RAPPORT DE PROJET

# Analyse du risque des transactions blockchain du métavers

Application d’apprentissage automatique avec Python et Streamlit

*Version 2 : prétraitement anti-fuite, évaluation rigoureuse, application refondue*

**Bouzidi Mariam · Lajmi Sahar · Eya Bouhouch**

Octobre 2026

| Ce qui change dans la version 2 |
| --- |
| 1. **La variable risk_score est retirée des entrées** : elle détermine l’étiquette à elle seule (100 % d’exactitude avec un arbre sur cette unique variable). |
| 2. **Trois scénarios de variables** rendent la fuite visible : complet, sans risk_score, comportemental (scénario par défaut). |
| 3. **Prétraitement piloté par des mesures** : redondance, bruit et fuite sont détectés automatiquement, puis consignés. |
| 4. **Tout ce qui apprend est ajusté sur le jeu d’entraînement seul** (imputation, écrêtage, mise à l’échelle, encodage). |
| 5. **Évaluation honnête** : macro-F1, rappel par classe, courbes ROC et PR calculées sur les scores, comparaison à une référence naïve. |

# 1. Introduction

Ce projet développe une application Python (Streamlit) qui compare plusieurs classifieurs pour déterminer le niveau de risque d'une transaction blockchain dans le métavers ouvert : **risque faible  modéré** ou **élevé**. Le jeu de données contient 78 600 transactions décrites par 14 attributs (horodatage, montant, type de transaction, région, fréquence de connexion, durée de session, profil d'achat, groupe d'âge, score de risque, etc.).

La première version obtenait des exactitudes proches de 100 % pour plusieurs modèles. L'audit mené pour cette seconde version montre que ces scores mesuraient une **fuite d'information** et non la qualité des modèles. Le rapport documente donc (i) ce que les données contiennent réellement, (ii) un prétraitement conçu à partir de ces constats, (iii) l'application refondue et (iv) une évaluation qui ne se laisse pas tromper par le déséquilibre des classes.

# 2. Jeu de données et audit

## 2.1 Description

Aucune valeur manquante ni ligne dupliquée. Seules 1 161 adresses émettrices distinctes produisent toutes les transactions : les adresses sont des *identifiants* et non des variables explicatives (un modèle les mémoriserait). Les classes sont déséquilibrées : faible 80,8 %, modéré 11,0 %, élevé 8,3 %. Un modèle qui prédit toujours « risque faible » obtient donc déjà **0,808** d'exactitude ; l'exactitude seule est trompeuse.

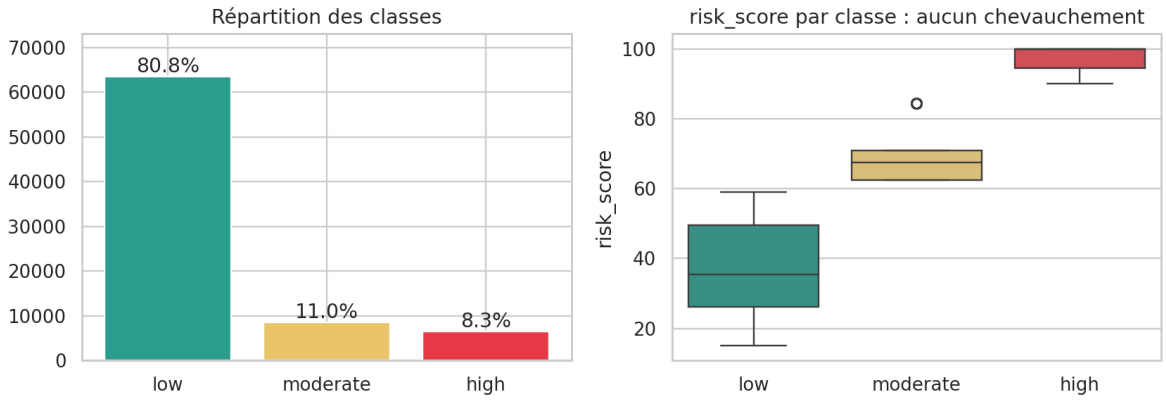



*Figure 1 : classes déséquilibrées (gauche) ; les plages de risk_score ne se chevauchent jamais (droite).*

## 2.2 Fuite d'information : risk_score

Les plages de `risk_score` par classe sont disjointes : 15 à 59,1 (faible), 62,5 à 84,4 (modéré), 90 à 100 (élevé). L'étiquette est un simple seuillage de ce score. Un test de fuite entraîne un arbre de profondeur 6 sur *une seule* variable à la fois (validation croisée à 3 plis) :

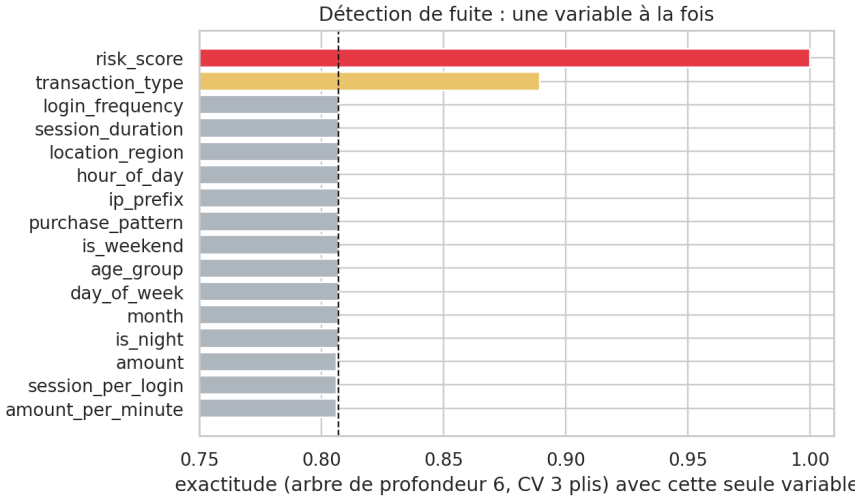



*Figure 2 : risk_score atteint 100 % seul ; transaction_type 88,9 %. La ligne pointillée est la référence « toujours faible ».*

## 2.3 Les étiquettes suivent des règles

Même sans `risk_score`, un arbre de profondeur 4 retrouve approximativement la structure : *scam* ou *phishing* → risque élevé ; *sale* pendant la nuit (0 h à 6 h) → risque modéré ; le reste → risque faible. Le jeu de données est **synthétique et généré par règles**. En conséquence, tout modèle disposant de `transaction_type` approche 100 %, et seule la suppression de ce raccourci permet de comparer réellement les algorithmes.

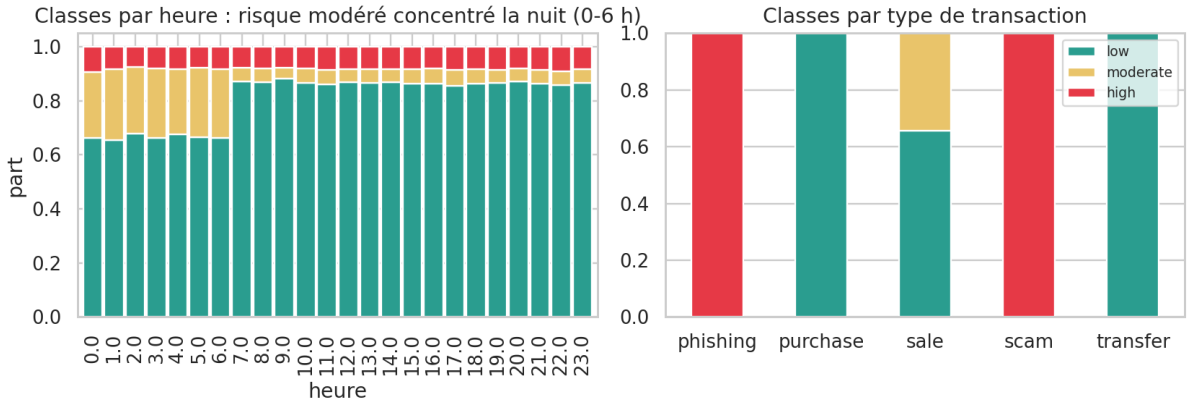



*Figure 3 : le risque modéré se concentre la nuit (gauche) ; phishing/scam sont toujours à risque élevé, purchase/transfer toujours à risque faible (droite).*

## 2.4 Redondance et variables sans signal

- **purchase_pattern** et **age_group** sont la même variable sous deux noms (veteran = high_value, new = random, established = focused ; V de Cramér = 1,0). Une seule est conservée : age_group, qui possède un ordre naturel.
- **location_region** (V = 0.007, p = 0.36) et **ip_prefix** (V = 0.007, p = 0.53) n'ont aucun lien mesurable avec le risque : supprimées automatiquement.
- Les variables calendaires (jour, mois, week-end) ont une information mutuelle nulle avec l'étiquette : exclues par défaut.
- Les comportements portent un vrai signal : l'information mutuelle de login_frequency (0.20) et session_duration (0.20) est la plus élevée après risk_score (0.62).

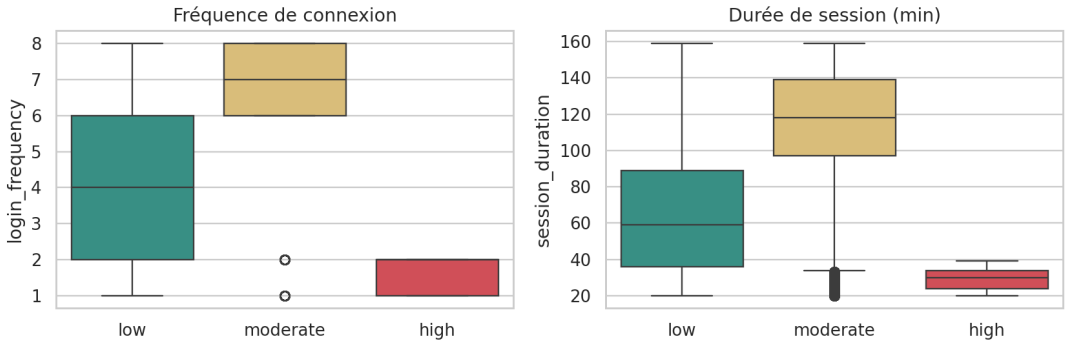



*Figure 4 : les profils à haut risque se connectent peu et restent peu de temps ; les profils modérés font l'inverse.*

# 3. Prétraitement

Le prétraitement est implémenté dans un module unique, `preprocessing.py`, utilisé à la fois par le notebook, l'application et l'inférence : les trois ne peuvent plus diverger. Chaque décision de suppression ou de transformation s'appuie sur une mesure de l'audit.

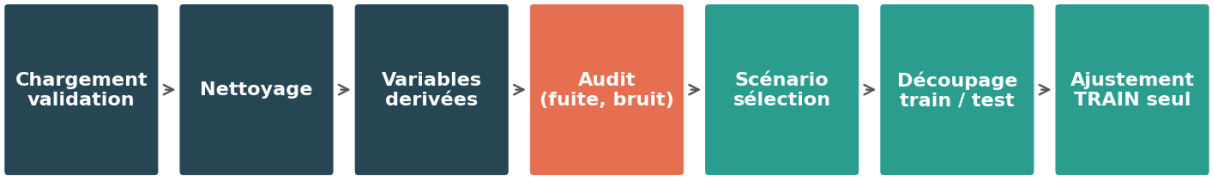



*Figure 5 : chaîne de traitement. Le découpage précède tout ajustement : le jeu de test n'influence aucun paramètre appris.*

## 3.1 Étapes

| Étape | Traitement | Justification |
|---|---|---|
| Nettoyage | Normalisation du texte, conversion des types, valeurs hors plage → NaN, hour_of_day recalculée depuis l'horodatage, doublons supprimés, tri chronologique | Robustesse sur des exports plus sales ; tri indispensable pour le découpage temporel |
| Cible | low = 0 < moderate = 1 < high = 2 (ordre fixe) | Corrige la v1 où LabelEncoder triait par ordre alphabétique (high = 0, low = 1, moderate = 2) alors que les graphiques supposaient low, moderate, high |
| Variables dérivées | is_night (0 h–6 h), sin/cos de l'heure, session_per_login, amount_per_minute | Point de rupture à 7 h mesuré dans les données ; l'heure est cyclique ; les ratios améliorent la régression logistique d'environ 0,02 en macro-F1 |
| Audit | Test de fuite, V de Cramér, information mutuelle, sonde de règles | Les suppressions sont décidées par les mesures, pas par intuition |
| Imputation | Médiane (numérique), valeur la plus fréquente (catégoriel) | Apprise sur le train seul |
| Écrêtage | Quantiles 0,1 % – 99,9 % (QuantileClipper) | Neutralise les extrêmes (amount max 1 557 contre 1 281 au quantile 99,9 %) ; amount est quasi symétrique (asymétrie 0,12), donc pas de log |
| Échelle | StandardScaler (ou MinMax au choix) | Nécessaire pour KNN, SVM, réseau de neurones, régression logistique |
| Encodage | One-hot (catégories inconnues tolérées) ; age_group ordinal new < established < veteran | Respecte la nature des variables |
| Déséquilibre | class_weight = balanced (priors uniformes pour Naive Bayes) | Poids des classes : 0: 0,41, 1: 3,04, 2: 4,03 |
| Découpage | Stratifié (70/30) ou temporel (passé → futur) | Le découpage temporel vérifie que le résultat n'est pas un artefact du tirage aléatoire |

## 3.2 Variables retenues et supprimées

| Colonne | Décision | Raison |
|---|---|---|
| sending_address, receiving_address | supprimées | identifiants : mémorisation des utilisateurs |
| timestamp | remplacée | variables temporelles dérivées |
| purchase_pattern | supprimée | doublon de age_group (V = 1,0) |

| Colonne | Décision | Raison |
|---|---|---|
| location_region, ip_prefix | supprimées | aucune association avec la cible |
| risk_score | retirée (scénarios 2 et 3) | fuite : détermine l'étiquette |
| transaction_type | retirée (scénario 3) | quasi-étiquette : phishing/scam → élevé |

Dans le scénario comportemental, la matrice finale compte **10 variables** : amount, login_frequency, session_duration, hour_of_day, amount_per_minute, session_per_login, hour_sin, hour_cos, is_night et age_group.

### 3.3 Trois scénarios

| Scénario | Variables retirées | Rôle |
|---|---|---|
| Complet | aucune | démontre la fuite ; ne pas utiliser pour conclure |
| Sans risk_score | risk_score | référence réaliste mais quasi triviale (transaction_type détermine presque tout) |
| **Comportemental** (par défaut) | risk_score, transaction_type | problème réellement difficile où les modèles diffèrent |

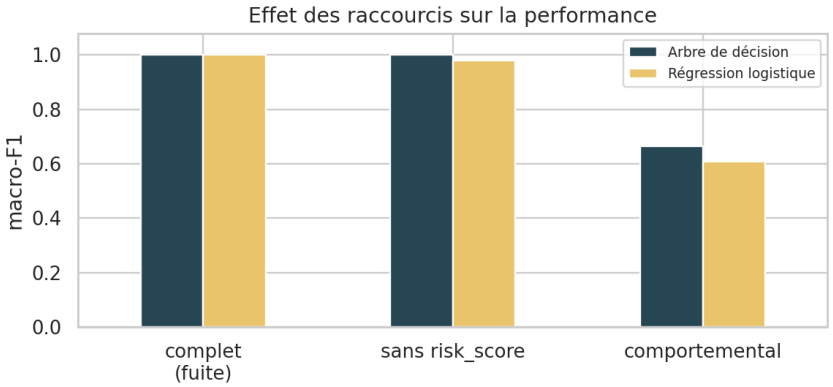



*Figure 6 : la même évaluation sur les trois scénarios. Les scores parfaits disparaissent dès que les raccourcis sont retirés.*

## 4. Application Streamlit

L'application est organisée en cinq onglets, avec un panneau latéral pour choisir le scénario, la stratégie de découpage, la taille du test et les options de prétraitement avancées. Un bandeau coloré rappelle en permanence si le scénario choisi comporte une fuite.

| Onglet | Contenu |
|---|---|
| Données et prétraitement | Indicateurs clés, étapes appliquées, colonnes supprimées et raisons, test de fuite, information mutuelle, sonde de règles, aperçus et téléchargement du jeu traité et du rapport |
| Explorer | Distributions par classe, répartition des classes par heure ou par catégorie, matrice de corrélation de Spearman |
| Entraîner un modèle | 8 classifieurs ; hyperparamètres manuels ou **réglage automatique** (recherche aléatoire, validation croisée, macro-F1) ; matrice de confusion, rapport par classe, ROC, précision-rappel, importance par permutation |
| Comparer les modèles | Classement sur macro-F1 avec référence naïve, export CSV, **laboratoire de fuite** sur les trois scénarios |
| Prédire une transaction | Formulaire (date, heure, montant, connexions, durée, profil) ; classe prédite, probabilités, indicateurs dérivés |

Améliorations techniques par rapport à la v1 : courbes ROC et PR calculées sur les *scores* (la v1 utilisait des prédictions binarisées, ce qui produit des courbes sans sens) ; courbe PR indépendante

de la ROC (elle plantait si la ROC n'était pas cochée) ; espaces de recherche réduits (la grille de la forêt aléatoire comptait environ 196 000 combinaisons × 5 plis) ; SVM entraîné sur un échantillon car sa complexité est quadratique ; cache des étapes coûteuses ; état conservé entre les interactions.

# 5. Modèles

| Modèle | Hyperparamètres exposés | Espace du réglage automatique |
|---|---|---|
| Random Forest | arbres, profondeur, feuille min., features par split | arbres {100, 200}, profondeur {6, 10, 14, 20}, feuille min. {1, 3, 5, 10} |
| Decision Tree | profondeur, split min., feuille min., critère | profondeur {3…16}, feuille min. {1…50}, gini/entropie |
| Gradient Boosting (HistGB) | tours, taux d'apprentissage, profondeur, L2 | taux {0,03…0,2}, profondeur {3, 5, 8}, tours {100, 200, 300} |
| Régression logistique | C, itérations max. | C {0,01…100} |
| K plus proches voisins | k, pondération, distance (L1/L2) | k {3…41}, pondération, L1/L2 |
| SVM | C, noyau (rbf/linéaire), gamma, taille d'échantillon | C {0,1…100}, gamma {scale, auto} |
| Réseau de neurones | couches, activation, alpha, taux, époques | couches {(32,)…(128, 64)}, alpha, taux, activation |
| Naive Bayes gaussien | lissage de variance | lissage $\{10^{-11}…10^{-1}\}$ |

Le Naive Bayes multinomial de la v1 exigeait des valeurs positives ; il est remplacé par un Naive Bayes **gaussien**, adapté à des variables continues standardisées. Les modèles qui le permettent utilisent des poids de classe équilibrés ; KNN et le réseau de neurones n'offrent pas cette option dans scikit-learn, ce qui explique une partie de leur comportement décrit plus bas.

# 6. Résultats

## 6.1 Protocole

Scénario comportemental, découpage stratifié 70/30 (graine 42), paramètres par défaut, métrique principale : **macro-F1** (moyenne non pondérée du F1 des trois classes). L'exactitude est rapportée mais ne sert pas au classement, car prédire toujours « faible » donne déjà 0,808 d'exactitude et seulement 0,298 de macro-F1.

| Modèle | Exactitude | Macro-F1 | Rappel macro | AUC macro |
|---|---|---|---|---|
| **Decision Tree** | 0,710 | **0,663** | 0,862 | 0,908 |
| Random Forest | 0,713 | 0,662 | 0,857 | 0,911 |
| Gradient Boosting | 0,711 | 0,660 | 0,860 | 0,911 |
| SVM (RBF) | 0,693 | 0,642 | 0,858 | 0,888 |
| Régression logistique | 0,664 | 0,609 | 0,843 | 0,892 |
| Réseau de neurones | 0,866 | 0,572 | 0,623 | 0,912 |
| KNN (k = 15) | 0,858 | 0,570 | 0,607 | 0,904 |
| Naive Bayes gaussien | 0,543 | 0,501 | 0,793 | 0,841 |
| *Référence : toujours « faible »* | *0,808* | *0,298* | – | – |

Tableau : résultats sur le jeu de test (23 580 transactions), scénario comportemental.

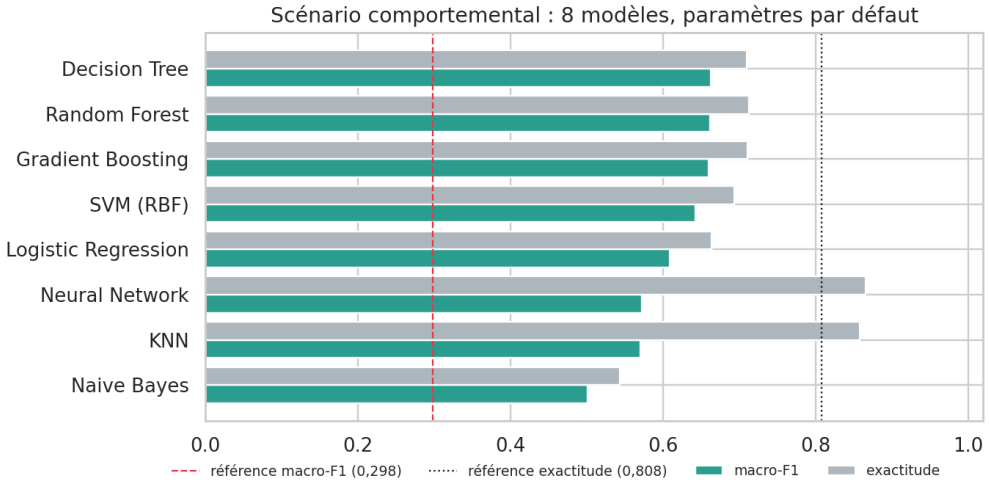



*Figure 7 : classement. L'exactitude (gris) et le macro-F1 (vert) donnent des ordres opposés.*

## 6.2 Lecture des résultats

- **L'exactitude classe à l'envers.** Le réseau de neurones et KNN affichent 0,86–0,87 d'exactitude mais seulement 0,57 de macro-F1 : ils prédisent presque toujours « faible » et ratent les classes minoritaires (rappel macro d'environ 0,61–0,62). Les modèles à arbres, pondérés, perdent en exactitude mais retrouvent 86 % de rappel macro.
- **Plafond autour de 0,66.** Arbre, forêt et boosting convergent vers le même macro-F1 ; le réglage automatique n'apporte qu'environ 0,02 au maximum (forêt : 0,662 → 0,665 ; KNN : 0,570 → 0,594). L'information comportementale disponible est donc le facteur limitant, non l'algorithme.
- **Résultat stable dans le temps.** Avec un découpage temporel (entraînement sur le passé, test sur le futur), l'arbre obtient 0,709 d'exactitude et 0,656 de macro-F1, contre 0,710 et 0,663 en découpage stratifié.

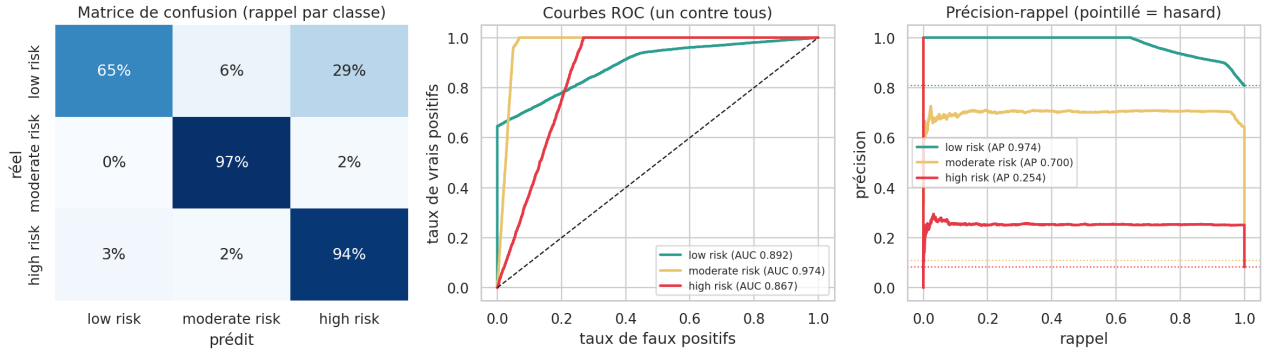



*Figure 8 : forêt aléatoire, scénario comportemental. Matrice normalisée par classe, ROC et précision-rappel calculées sur les probabilités.*

## 6.3 Limite structurelle : la précision de la classe « élevé »

Pour la forêt aléatoire, le rappel du risque élevé est de 0,944 mais sa **précision n'est que de 0,251** (AP 0,254). Les comptes « new » représentent 33 % des transactions et 25 % d'entre eux seulement sont à haut risque : le comportement observé ne suffit pas à les départager de comptes à faible risque. À l'inverse, la classe modérée est bien identifiée (rappel 0,973, AUC 0,974). Pour un usage réel, il faudrait choisir le seuil de décision selon le coût d'une fausse alerte face à celui d'un risque manqué.

# 7. Corrections apportées à la version 1

| Problème constaté en v1 | Conséquence | Correction en v2 |
|---|---|---|
| risk_score utilisé comme variable | exactitudes de 100 % sans valeur | scénarios ; retiré par défaut ; test de fuite documenté |
| Ordre alphabétique du LabelEncoder | matrice de confusion étiquetée de travers | ordre fixe low < moderate < high |
| purchase_pattern et age_group tous deux encodés | information dupliquée | détection automatique (V = 1,0) |
| Rapport (RF 0,89 ; NB 0,78) incohérent avec l'application (1,0 / 0,99) | résultats non reproductibles | tous les chiffres proviennent de l'exécution du code ci-joint |
| Mise à l'échelle et encodage sur toutes les données | fuite train → test | tout est ajusté sur le train seul (ColumnTransformer) |
| ROC calculée sur des prédictions binaires ; PR dépendante de la ROC | courbes erronées ; plantage | scores continus ; courbes indépendantes |
| Exactitude seule, pas de référence naïve | modèles mal classés | macro-F1, rappel par classe, référence « toujours faible » |
| Grille de recherche d'environ 196 000 combinaisons (RF) | application inutilisable | recherche aléatoire sur espaces réduits |
| Naive Bayes multinomial sur variables standardisées | valeurs négatives interdites | Naive Bayes gaussien |
| Chemin de fichier absolu (C:/Users/...) dans le notebook | non reproductible | chemins relatifs, module partagé |

# 8. Limites et perspectives

- **Données synthétiques.** Les règles qui génèrent les étiquettes limitent la portée des conclusions ; les performances ne se transposent pas telles quelles à des transactions réelles.
- **Une seule source de données.** L'application fonctionne sur ce jeu ; le chargement de fichiers utilisateur (avec validation de schéma) est la suite naturelle.
- **Seuil de décision.** Ajouter un curseur de seuil et une matrice de coûts pour arbitrer fausses alertes et risques manqués.
- **Explicabilité.** Ajouter SHAP en complément de l'importance par permutation.
- **Validation externe** sur un jeu réel ou sur un second jeu synthétique généré selon d'autres règles.

# 9. Conclusion

Cette seconde version transforme un classement de modèles aux scores artificiellement parfaits en une étude de **fuite d'information et d'évaluation honnête**. Le principal enseignement est méthodologique : sur ce jeu, la variable risk_score et le type de transaction suffisent à retrouver l'étiquette, de sorte que seule une configuration comportementale permet de comparer les algorithmes. Dans cette configuration, les modèles à arbres pondérés atteignent un macro-F1 d'environ 0,66 (rappel macro de 0,86), alors que les modèles les plus « exacts » ne font que reproduire la classe majoritaire. L'application permet de refaire chacune de ces observations de manière interactive.

# Annexe : exécution

```
pip install -r requirements.txt
streamlit run app.py
```

```
python preprocessing.py --scenario behavioural --out artifacts
```

Fichiers : `app.py` (interface), `preprocessing.py` (pipeline, audit, inférence), `preprocessing.ipynb` (analyse commentée), `metaverse_transactions_dataset.csv`.